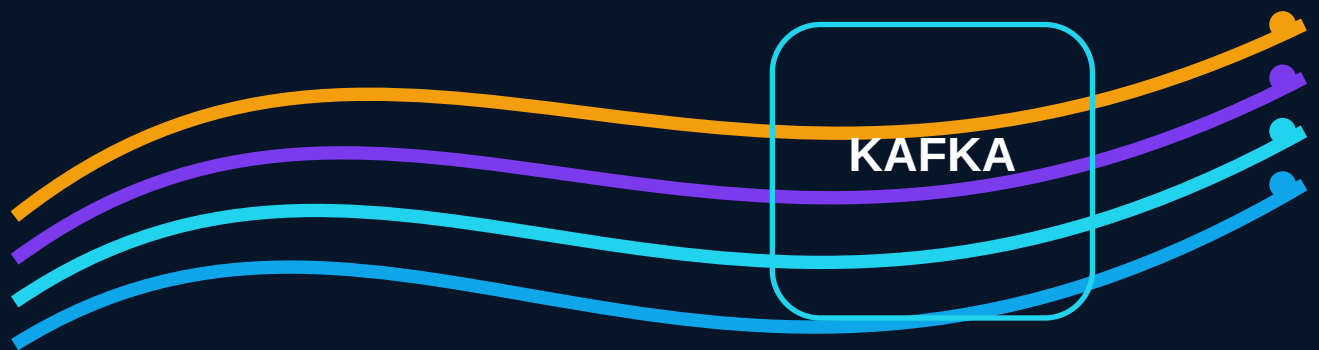


Apache Kafka

Master Knowledge Guide

A deep learning resource for system design, interviews, and production thinking. Built for CodeAsura style: mental models, diagrams, real-world stories, and memory hooks.



How to use this guide

This PDF is not a video script. It is the master knowledge base behind the Kafka video series.

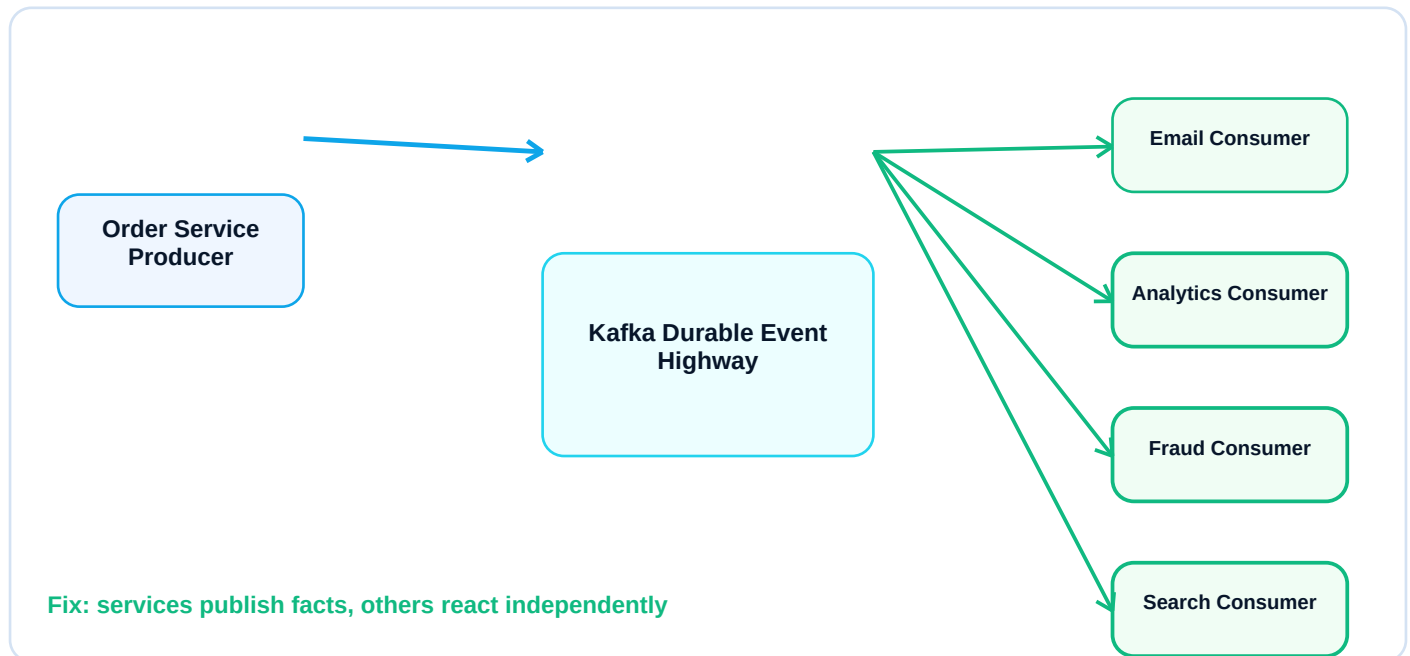
Use it for	How to read
First-time learning	Read Sections 1-7 in order. Focus on mental models before configs.
Interview prep	Use the Q&A pages, decision tables, and system design script.
Production work	Study sections on producer guarantees, consumers, operations, security, and failure modes.
YouTube audience download	Use cheat cards and diagrams for quick revision.

REMEMBER

Your memory chain: Event -> Topic -> Partition -> Broker -> Consumer -> Offset -> Lag -> Rebalance -> Failure mode.

Kafka in one mental model

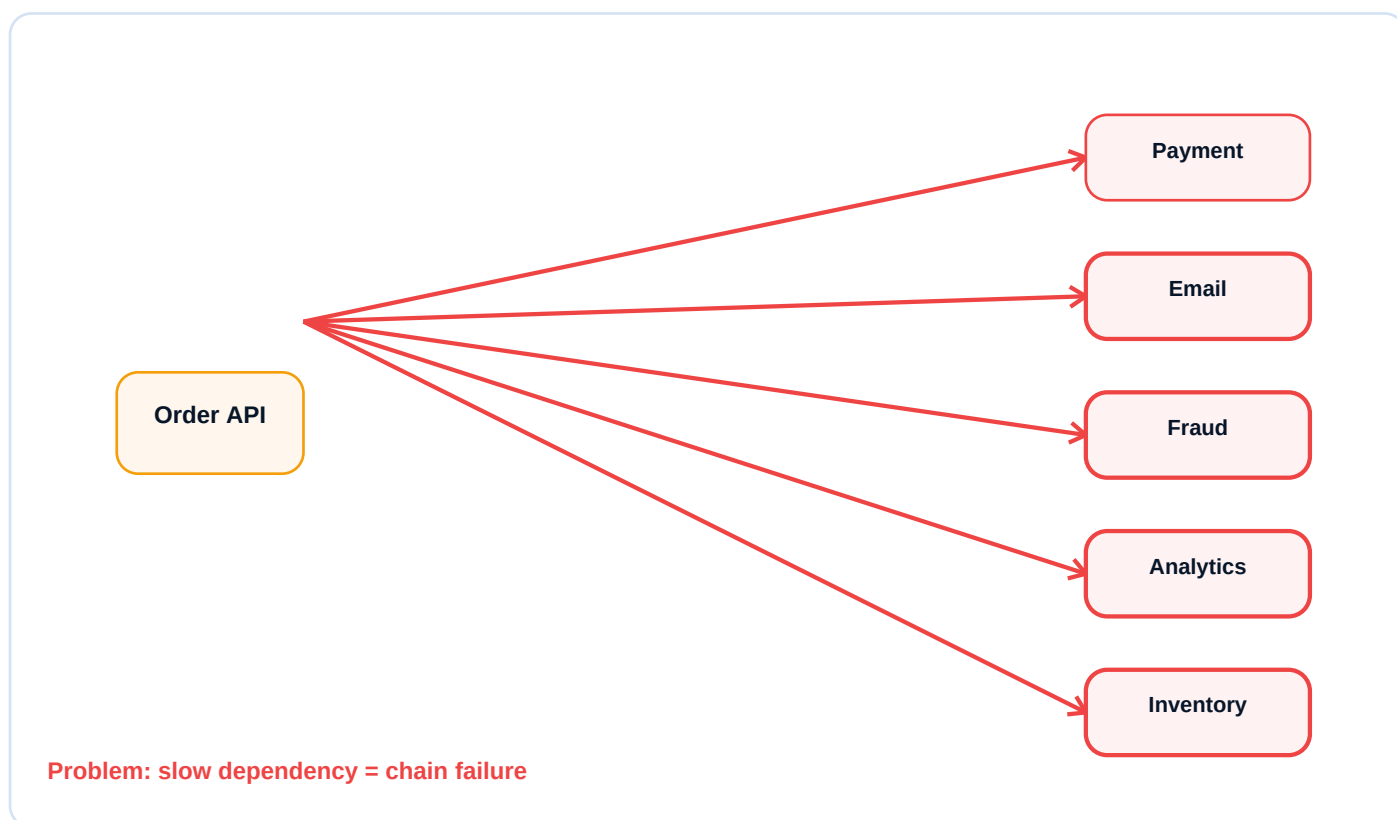
Kafka is a durable event highway. Producers put facts onto named streams. Kafka stores those facts. Consumers read them independently, now or later.

**REMEMBER**

Kafka is not magic glue. It is durable, partitioned, replicated event storage with a pub-sub API.

Why direct service calls break

Small systems can call each other directly. Large systems become a spider web of synchronous dependencies.



- Every new dependency increases latency and failure paths.
- One slow downstream can block the user-facing flow.
- Retries from many services can amplify an outage.
- Teams become tightly coupled because one change affects many services.

REMEMBER

Kafka does not replace APIs. Kafka removes unnecessary synchronous coupling where async events make more sense.

The failure story: order checkout

A checkout service directly calls payment, inventory, email, analytics, fraud, shipping, and search indexing.

Dependency	If it is slow...	Better with Kafka?
Payment	User cannot complete checkout	Usually keep sync for immediate payment result.
Email	Checkout should not wait	Publish OrderPlaced; email consumes later.
Analytics	Business dashboards lag but user flow should not fail	Publish event stream.
Fraud	Sometimes sync for hard block, async for scoring/auditing	Hybrid.
Search index	Index can update seconds later	Async consumer.

REMEMBER

Good system design is not "Kafka everywhere". It is choosing what must be synchronous and what can be event-driven.

Kafka is not just a queue

Kafka can act like a queue in some patterns, but its real power is durable, replayable event streams.

Pattern	Mental model	Message lifetime	Best for
Queue	Work line	Often removed after consume	Task distribution
Pub-sub broker	Broadcast station	Often transient	Fan-out notifications
Kafka	Durable event log	Stored by retention/compaction	Streams, replay, decoupling, analytics
Database	Current state store	Mutable rows/documents	Queries and transactions

REMEMBER

Queue mindset: "deliver and delete". Kafka mindset: "append, store, replay, process".

Core vocabulary: event, record, key, value

Kafka moves records. In product language, those records usually represent business events.

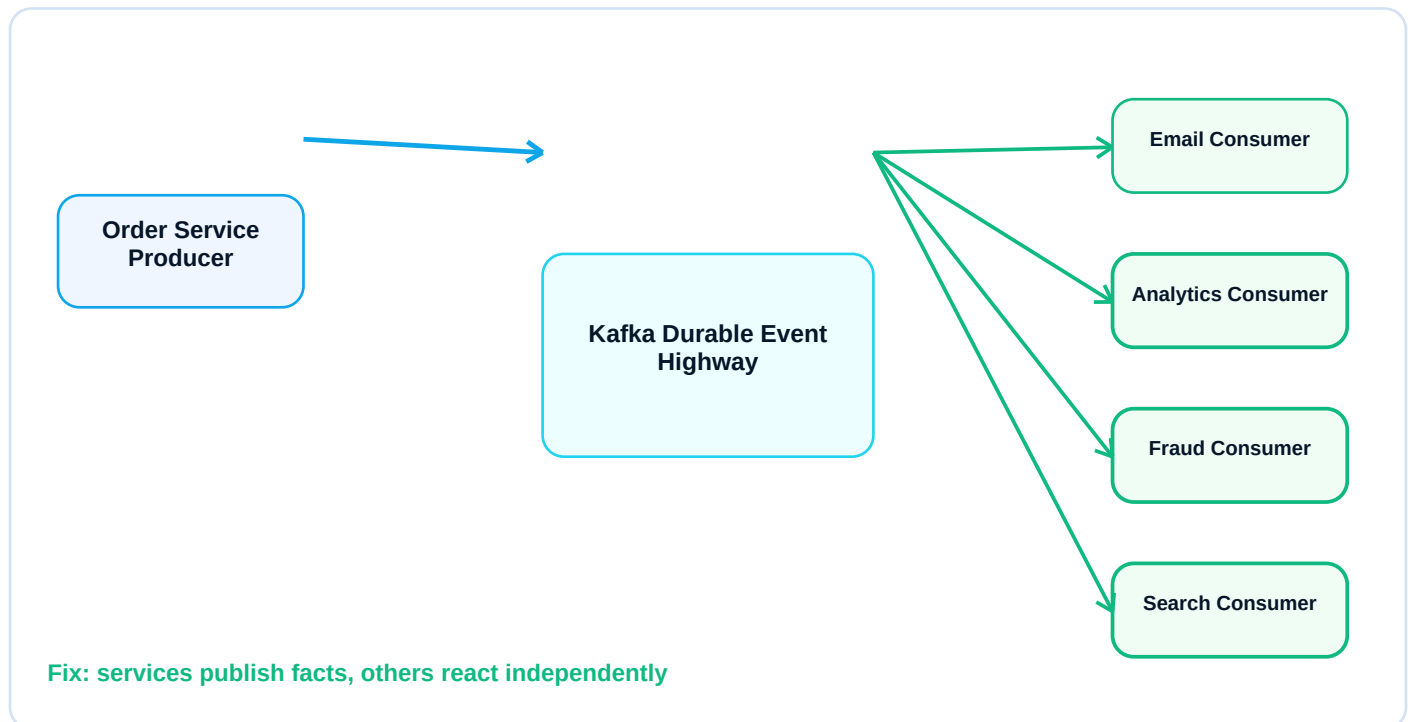
Term	Meaning	Example
Event	A fact that something happened	OrderPlaced
Record/message	Kafka unit stored in a topic partition	{key, value, timestamp, headers}
Key	Used for partition routing and ordering per entity	orderId or userId
Value	Payload of the event	order total, items, status
Headers	Metadata for routing, trace IDs, schema info	correlation-id
Timestamp	Event/create time or append time	2026-05-10T10:15:00Z

REMEMBER

An event should be named as a past-tense fact: **OrderPlaced**, **PaymentCaptured**, **UserSignedUp**.

Kafka architecture overview

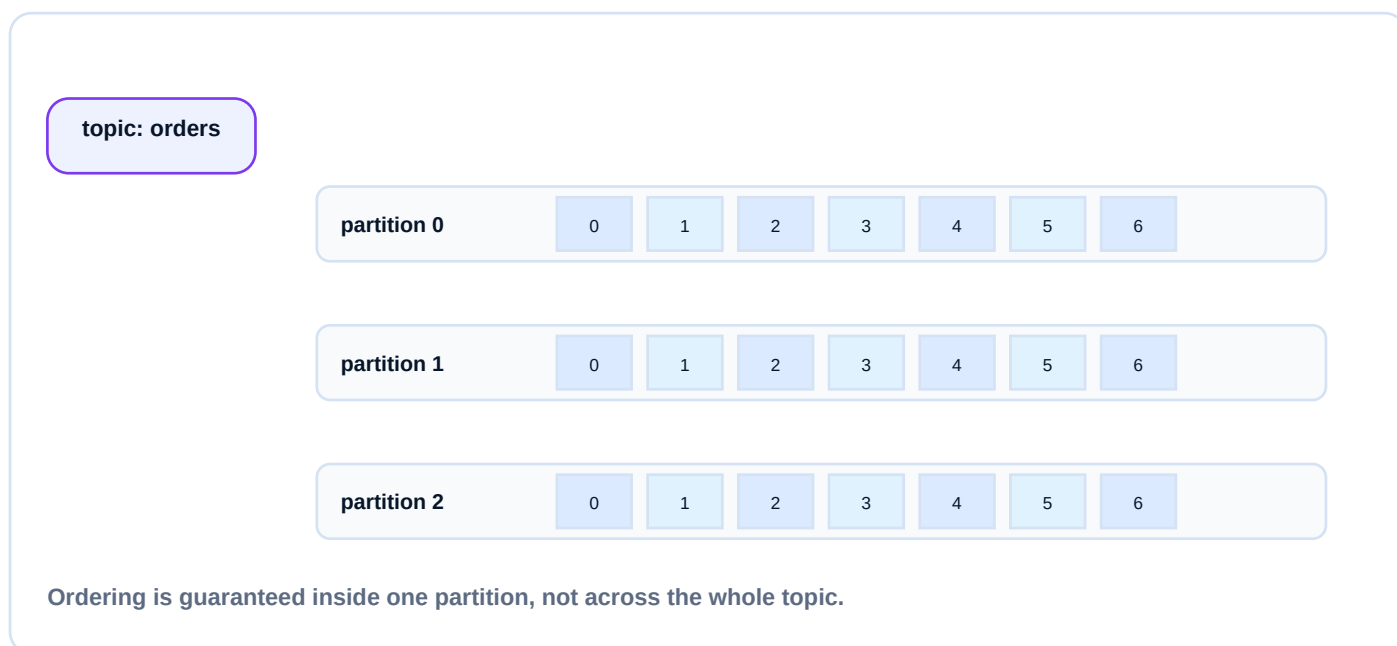
The simple view: producers write to topics, brokers store topic partitions, consumers read at their own speed.



- Producer: application that writes records.
- Topic: named stream/category of records.
- Partition: ordered shard of a topic.
- Broker: Kafka server that stores partitions.
- Consumer: application that reads records.
- Cluster: multiple brokers working together.

Topics and partitions

A topic is split into partitions so Kafka can scale storage, writes, reads, and parallel processing.



- A record belongs to exactly one partition within a topic.
- Records are appended to the end of a partition log.
- More partitions allow more parallelism, but also more operational overhead.
- Ordering is only guaranteed within a single partition.

REMEMBER

If order matters for a user/order/account, use the same key so related events land in the same partition.

Partition key strategy

The key decides where a record goes. Bad keys create hot partitions or broken ordering.

Goal	Key choice	Risk
Keep per-user order	userId	Celebrity user can create hot partition
Keep per-order order	orderId	Good for order lifecycle events
Spread load evenly	random/no key	No ordering for related records
Tenant isolation	tenantId or tenantId+entityId	Large tenant may become hot
Payment correctness	paymentId/accountId	Must align with idempotency design

REMEMBER

Key design is system design. It controls ordering, load distribution, and failure blast radius.

Kafka ordering: the rule people forget

Kafka preserves order inside a partition. It does not automatically preserve global order across partitions.

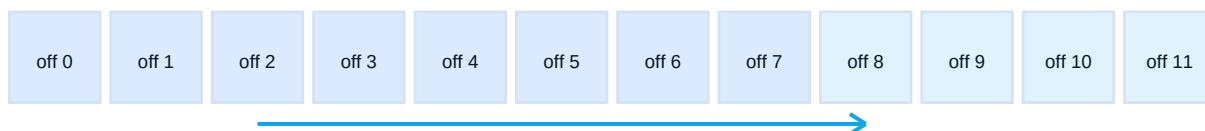


Question	Answer
Can Kafka preserve order?	Yes, within one partition.
Can one topic have global order across many partitions?	Not without sacrificing parallelism or custom ordering logic.
How to keep all events for one entity ordered?	Use a stable key for that entity.
What breaks ordering?	Wrong key, parallel handling per entity, external side effects, retries without idempotency.

Broker and storage model

A Kafka broker stores partition replicas on disk. Kafka is fast because it appends sequentially and serves consumers by offset.

Partition log on disk



Consumers ask: give me records from offset N

- Kafka writes records to partition logs.
- Records are not immediately deleted when one consumer reads them.
- Retention policy decides how long data remains.
- Consumers track offsets to know where to resume.

Retention and compaction

Kafka stores events based on topic policy. Two common modes are time/size retention and log compaction.

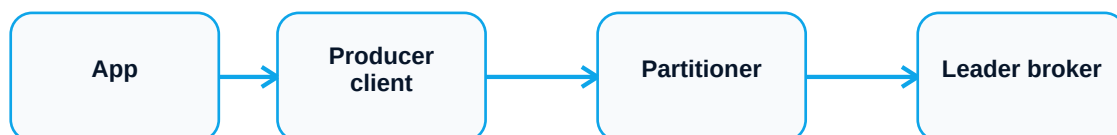
Mode	What it keeps	Use case	Watch out
Retention	Records for X time or size	Audit, event replay, data pipelines	Storage grows with throughput
Compaction	Latest value per key eventually	State snapshots, user profile, config changes	Old versions may vanish after compaction
Delete + compact	Hybrid policy	State topics with bounded storage	Requires careful expectation setting

REMEMBER

Retention answers: how far back can I replay? Compaction answers: what is the latest value per key?

Producer path: what happens on write

A producer batches records, picks a partition, sends to the leader broker, and receives an acknowledgement based on acks settings.



Batches and compression are key for throughput. acks decides durability level.

- Batching improves throughput but adds small latency.
- Compression reduces network/disk cost, often helpful for large payload volume.
- acks=all with replication is the safer default for important data.
- Retries require idempotence to avoid duplicate writes.

Producer guarantees: acks, retries, idempotence

Most correctness problems start with misunderstanding producer acknowledgements and retries.

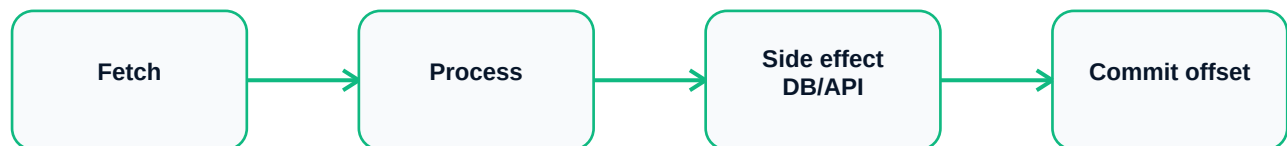
Setting	Meaning	Use carefully
acks=0	Producer does not wait for broker acknowledgement	Fast but can lose data silently
acks=1	Leader confirms write	Can lose data if leader dies before replication
acks=all	Leader waits for in-sync replicas based on min.insync.replicas	Safer, higher latency
retries	Producer can retry transient failures	Can create duplicates without idempotence
enable.idempotence	Producer avoids duplicate writes from retries per producer session	Not the same as consumer idempotency

REMEMBER

Safe producer baseline for important data: acks=all, idempotence on, meaningful retries, and monitored error handling.

Consumer path: pull, process, commit

Kafka consumers pull records from brokers. The consumer decides when it has safely processed a record and can commit progress.



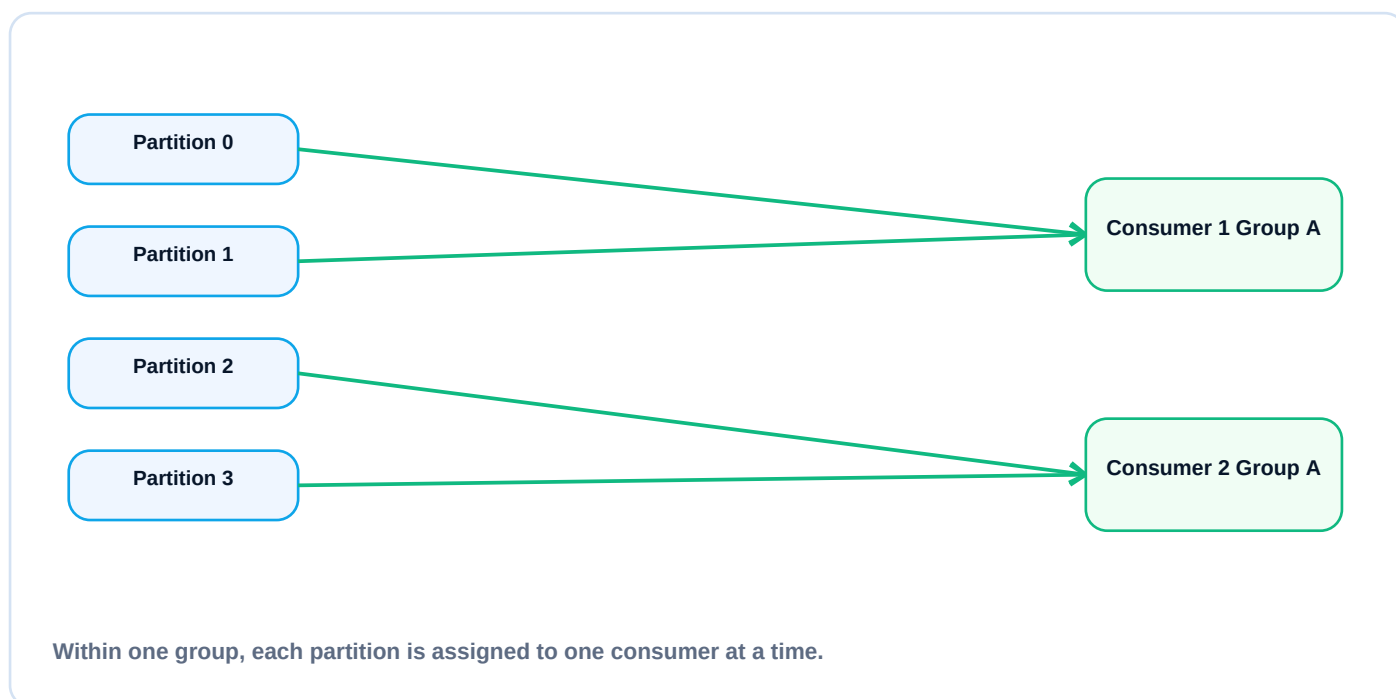
Commit too early -> data loss. Commit too late -> duplicate processing after crash.

REMEMBER

Kafka gives you the record again if progress was not committed. Your consumer must be idempotent.

Consumer groups

A consumer group is a team of consumers sharing work for the same topic. Partitions are assigned across group members.

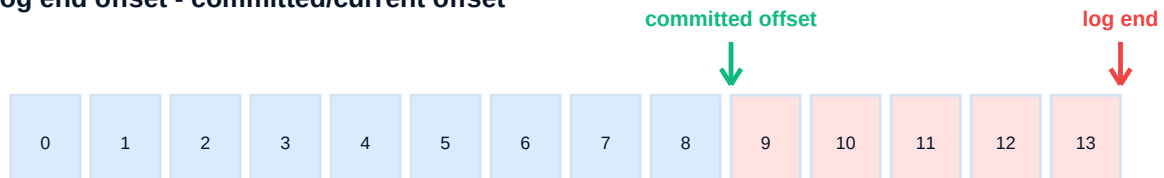


- If a topic has 4 partitions and a group has 2 consumers, each consumer gets some partitions.
- If consumers exceed partitions, extra consumers are idle.
- Different consumer groups can independently read the same topic from different offsets.
- Consumer groups are how Kafka supports both queue-like work sharing and pub-sub fanout.

Offsets and lag

Offsets are progress markers. Lag tells you how far behind a consumer is.

Lag = log end offset - committed/current offset



Metric	Meaning	Why it matters
Current offset	Position consumer has reached	Shows processing progress
Committed offset	Position saved for recovery	Determines restart point
Log end offset	Newest record in partition	Producer side growth
Lag	Records behind log end	Consumer health and scaling signal

REMEMBER

Lag is not always bad. Bad lag is lag that keeps increasing when business needs low latency.

Rebalancing: the hidden production pain

A rebalance redistributes partitions among consumers when group membership or metadata changes.

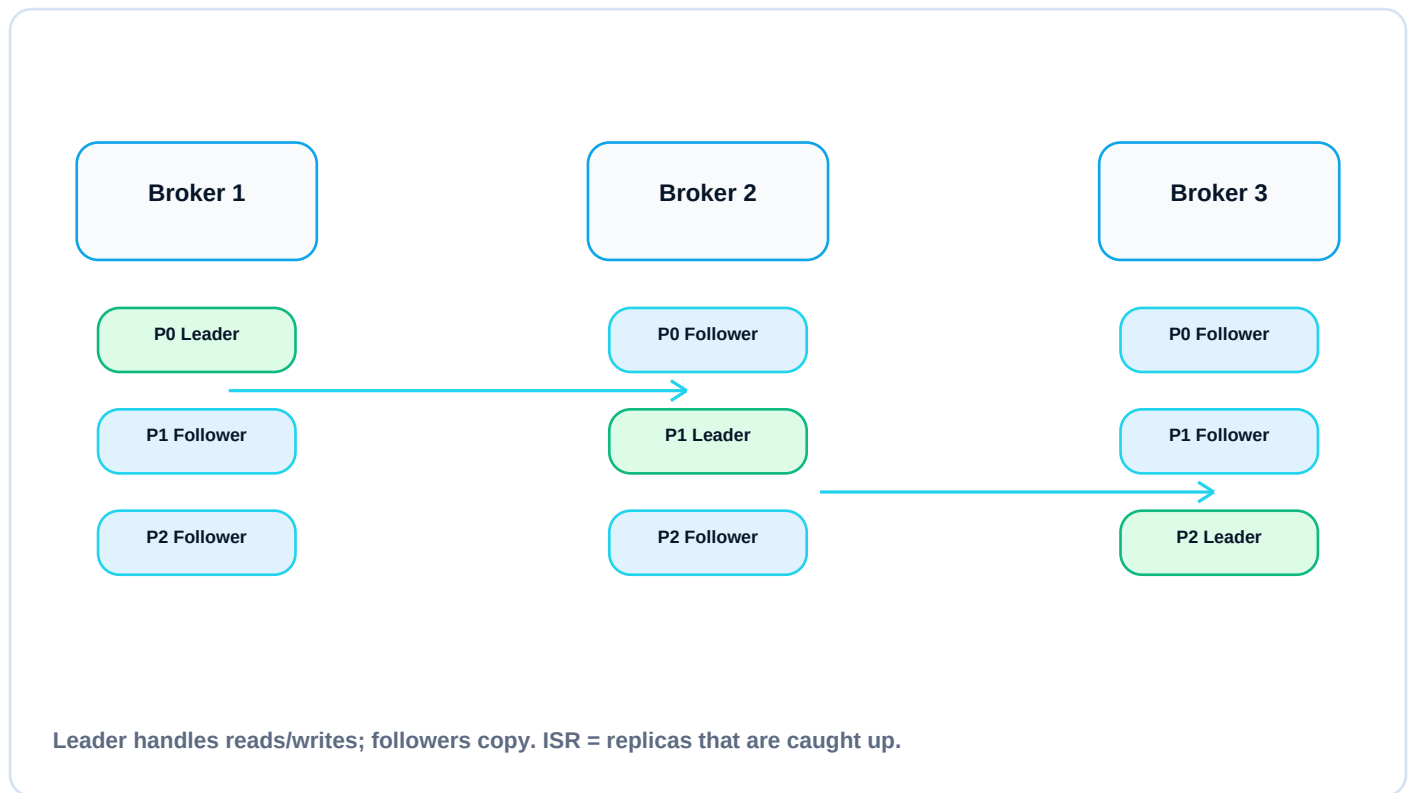
Trigger	Effect	Mitigation
Consumer joins/leaves	Partitions move to other consumers	Stable membership, graceful shutdown
Consumer too slow	Session timeout, partition revoked	Tune max.poll.interval.ms, optimize processing
Topic partition change	Assignments recalculated	Plan partition changes carefully
Crash/restart loop	Repeated rebalances and duplicates	Fix crash root cause, use DLQ/idempotency

REMEMBER

Rebalancing is not just a Kafka concept. It is where duplicates, pauses, and interview follow-up questions appear.

Replication: leaders, followers, ISR

Kafka replicates partition data across brokers. One replica is leader; followers copy it.



- Producer writes go to the partition leader.
- Followers replicate from the leader.
- ISR means in-sync replicas, replicas caught up enough to be considered safe.
- If a leader fails, an eligible follower can become leader.

Durability settings that matter

Kafka durability is not one switch. It is a combination of replication and acknowledgement settings.

Setting	What it controls	Practical advice
replication.factor	Number of copies for a partition	Usually 3 for production critical topics
min.insync.replicas	Minimum ISR needed for successful acks=all write	Commonly 2 with RF=3
acks=all	Producer waits for required ISR acknowledgement	Use for critical data
unclean.leader.election.enable	Whether out-of-sync replica can be leader	Usually false for correctness
retention.ms/bytes	How long data is available	Match replay/audit needs

REMEMBER

Interview line: durability requires RF, ISR, acks, and operational monitoring - not just "Kafka replicates data".

Delivery semantics without lies

Most Kafka confusion comes from mixing producer delivery, Kafka storage, and consumer side effects.

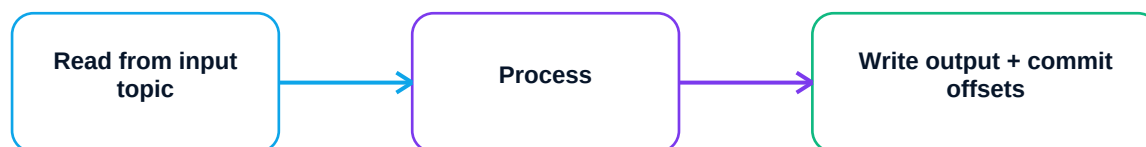
Semantic	Meaning	Typical trade-off
At most once	May lose, no duplicate	Fast, unsafe for important data
At least once	No loss if configured correctly, duplicates possible	Common practical default
Exactly once	No duplicate effects within a correctly designed Kafka transaction pipeline	Complex, not magic for external systems
Effectively once	Idempotent consumer makes duplicates harmless	Often best practical system design answer

REMEMBER

Exactly once does not mean the universe has no duplicates. It means a correctly scoped pipeline avoids duplicate effects.

Exactly-once: what to say in interviews

Kafka provides tools like idempotent producers and transactions, but your application still owns external side effects.



Transactions help commit output records and consumed offsets together inside Kafka.

- Use idempotent producer to avoid duplicate writes from retries.
- Use transactions for consume-transform-produce pipelines.
- For external DB/API side effects, use idempotency keys, unique constraints, or outbox/inbox patterns.
- Do not oversell exactly-once. Explain the boundary clearly.

Schema design and compatibility

Kafka decouples teams, but schemas keep them from accidentally breaking each other.

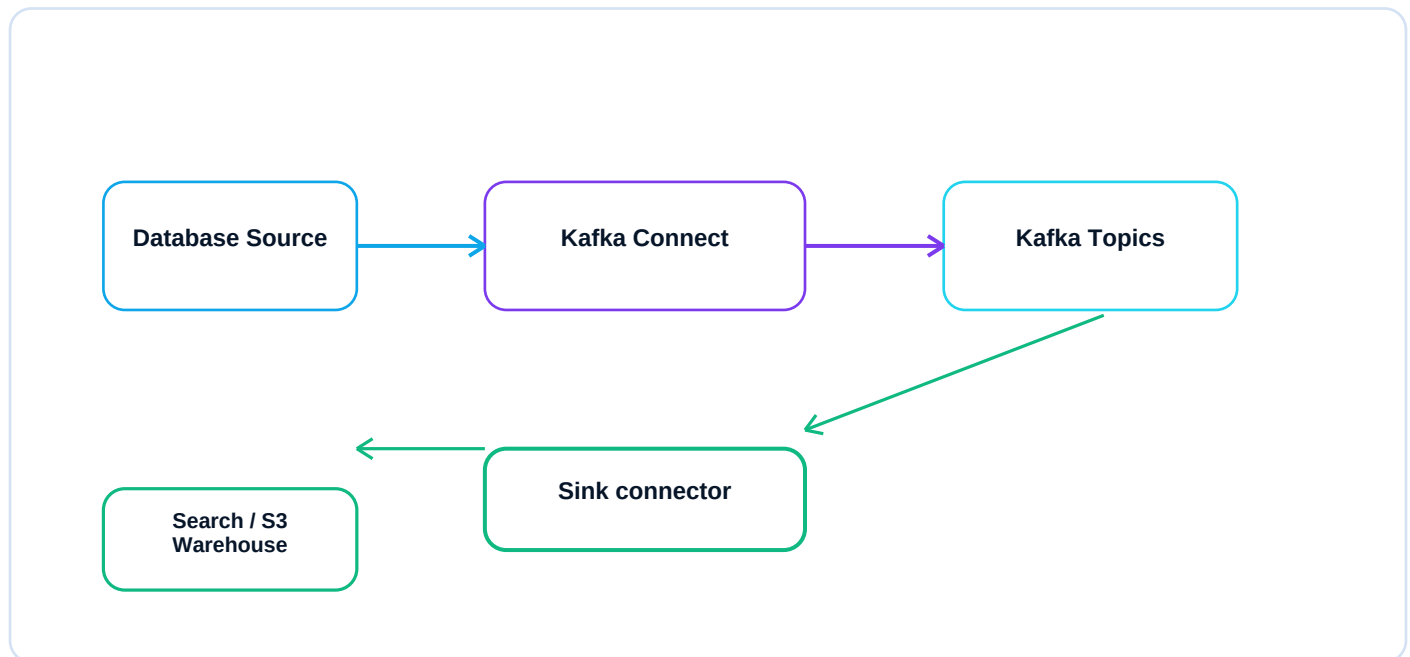
Problem	Bad outcome	Better practice
Producer removes a field	Consumer crashes	Use compatibility rules and schema registry
Event name is vague	Teams misunderstand meaning	Use clear past-tense names
Payload is too large	Broker/network/storage pressure	Store large blob elsewhere; send reference
No versioning	Hard migrations	Version schema and evolve safely
No ownership	Nobody maintains contract	Topic owner and data contract docs

REMEMBER

Events are contracts. Treat them like APIs: owned, versioned, documented, and monitored.

Kafka Connect: integration layer

Kafka Connect moves data between Kafka and external systems without writing custom producer/consumer code for every integration.



- Source connector: external system -> Kafka.
- Sink connector: Kafka -> external system.
- Great for CDC, analytics pipelines, search indexing, data lake ingestion.
- Still requires monitoring, schema management, retry/DLQ strategy, and connector tuning.

Kafka Streams: processing event streams

Kafka Streams is for building applications that transform, aggregate, join, and enrich Kafka streams.

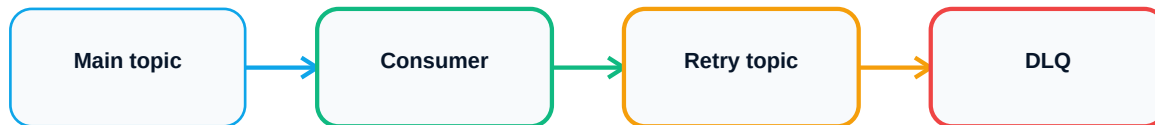
Operation	Example
Map/filter	Drop test events or transform payload shape
Aggregate/window	Count orders per 5 minutes
Join	Join clickstream with user profile updates
State store	Keep local state for fast stream processing
Exactly-once processing	Useful for Kafka-to-Kafka pipelines with transactions

REMEMBER

Streams is for continuous processing. Connect is for integration. Consumers are for application-specific processing.

Retries and dead letter queues

Production Kafka is not complete until you know what happens when a message fails.



Retry with backoff. Send poison messages to DLQ with context.

Failure type	Example	Treatment
Transient	Temporary DB timeout	Retry with backoff
Poison message	Invalid schema/business data	DLQ with error reason and original message
Slow dependency	Third-party API slow	Circuit breaker, retry topic, rate limit
Code bug	Consumer crashes repeatedly	Pause rollout, fix code, replay safely

Outbox pattern: DB + Kafka consistency

Classic problem: update database and publish Kafka event. What if one succeeds and the other fails?



Write business row and outbox row in one DB transaction. Publisher later sends to Kafka.

REMEMBER

Outbox turns "DB update + Kafka publish" into a recoverable workflow.

Kafka security layers

Security is not optional. Kafka often carries business-critical and personally sensitive event data.

TLS: encrypt wire

SASL / mTLS: prove identity

ACL: authorize action

Audit: trace access

- Encrypt traffic with TLS where needed.
- Authenticate clients and brokers using SASL, SSL/mTLS, or OAuth mechanisms depending on platform.
- Authorize actions with ACLs or platform IAM/RBAC controls.
- Avoid broad wildcard access in production. Audit who can read/write/admin each topic.

KRaft: Kafka without ZooKeeper

Modern Kafka uses KRaft mode for cluster metadata management. ZooKeeper mode has been removed in Kafka 4.0 and later.

Old model	Modern model
Kafka brokers + ZooKeeper ensemble	Kafka brokers/controllers using KRaft metadata quorum
Two systems to operate and secure	Kafka-native metadata management
ZooKeeper dependency in older clusters	KRaft required for Kafka 4.0+
Migration required before upgrading old ZooKeeper clusters	New deployments should use KRaft

REMEMBER

For interviews: mention ZooKeeper historically, but design new Kafka clusters with KRaft.

Sizing Kafka: a practical starting point

Kafka sizing depends on throughput, retention, replication, partitions, consumer parallelism, and failure tolerance.

Question	Why it matters
How many messages/sec and MB/sec?	Determines broker/network/disk capacity
How long must data be retained?	Determines storage
How many consumer groups?	Read load can be many times write load
What ordering key is required?	Determines partition key and parallelism limits
What is acceptable data loss?	Determines acks, RF, min ISR, replication
What is acceptable lag?	Determines consumer count and processing design

REMEMBER

Never choose partitions randomly. Choose them from throughput, ordering, and consumer parallelism.

Performance tuning mental model

Kafka throughput is usually limited by batching, compression, disk/network, partition count, and consumer processing speed.

Layer	Tune
Producer	batch.size, linger.ms, compression.type, acks, retries
Broker	disk IO, network, partition leadership balance, JVM/GC, page cache
Topic	partition count, replication factor, retention, compaction
Consumer	fetch size, concurrency model, max poll interval, commit strategy
Application	idempotency, DB writes, external calls, backpressure

REMEMBER

Most consumer lag is not Kafka being slow. It is the consumer doing slow work after fetch.

Observability: what to monitor

If you cannot see lag, ISR health, throughput, and errors, you cannot run Kafka safely.

Area	Metrics / signals
Broker health	Under-replicated partitions, offline partitions, ISR shrink/expand, controller health
Throughput	Bytes in/out, records in/out, request rate, produce/fetch latency
Consumer	Lag by group/topic/partition, rebalance rate, commit failures
Producer	Error rate, retry rate, request latency, batch size, record send rate
Storage	Disk usage, retention pressure, segment growth, disk IO
Business	Events/sec per domain, failed events, DLQ rate, end-to-end delay

REMEMBER

Alert on symptoms and causes: consumer lag, under-replicated partitions, broker disk, produce errors, DLQ spikes.

Production failure scenarios

Good Kafka engineers do not just know components. They know failure modes and recovery behavior.

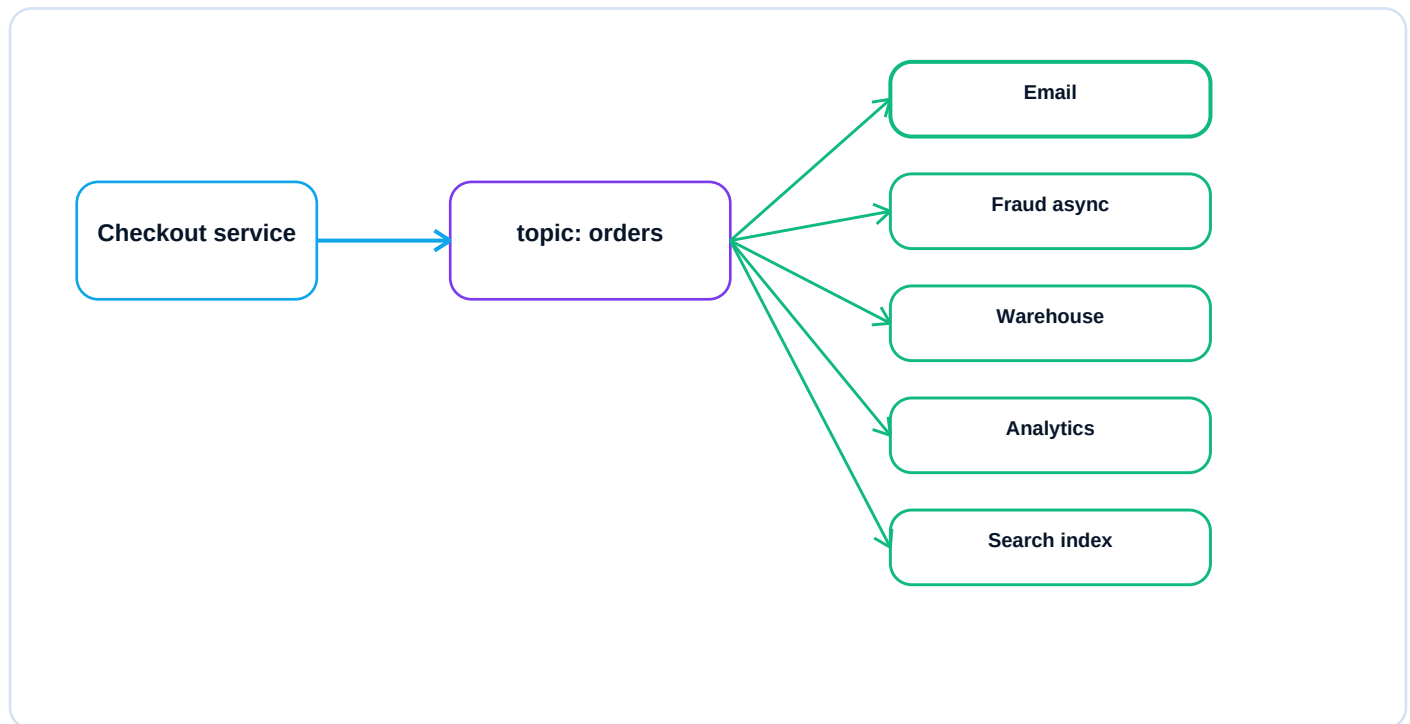
Failure	What happens	What to check
Broker dies	Leaders move if replicas are healthy	ISR, under-replicated partitions, controller logs
Consumer dies	Group rebalances; partitions reassigned	Consumer errors, commit point, duplicate effects
Producer retries	Can duplicate without idempotence	Producer configs and idempotency
Hot partition	One partition lags while others are fine	Key distribution, top keys, partition metrics
Schema change breaks consumer	Consumer errors or DLQ spike	Schema compatibility and rollout order
Disk full	Broker instability or write failures	Retention, partitions, disk expansion

REMEMBER

Interview gold: always mention what fails, how Kafka reacts, and what metric confirms the problem.

Real-world example: e-commerce order platform

Kafka fits when multiple downstream systems need the same business fact without blocking checkout.



- Synchronous: payment authorization and order confirmation.
- Asynchronous: email, analytics, indexing, warehouse notifications.
- Consumer failures should not fail checkout.
- Events allow replay if a downstream system was offline.

Real-world example: payment events

Payments need careful semantics. Kafka helps distribute facts, but idempotency and audits are mandatory.

Event	Producer	Consumers
PaymentAuthorized	Payment service	Order service, fraud, ledger, notification
PaymentCaptured	Payment service	Ledger, settlement, analytics, receipt
RefundIssued	Refund service	Ledger, notification, CRM
ChargebackOpened	Risk system	Fraud, support, account restriction

REMEMBER

Payment events must be immutable facts. Corrections should usually be new events, not mutation of old events.

Kafka system design answer script

Use this structure when an interviewer asks: "How would you design an event-driven system with Kafka?"

Step	Say this
1. Clarify	Which events? Throughput? Ordering? Retention? Latency? Loss tolerance?
2. Model events	Define event names, keys, schema, owners, and compatibility.
3. Topic design	Topics by domain stream, partitions by scale and ordering key.
4. Producer design	acks, retries, idempotence, batching, compression, error handling.
5. Consumer design	Groups, offsets, idempotent processing, retry/DLQ, lag monitoring.
6. Reliability	Replication, min ISR, retention, failover, replay plan.
7. Ops	Security, observability, alerts, capacity, runbooks.

REMEMBER

A senior answer mentions ordering, idempotency, lag, retention, failure mode, and observability.

Interview Q&A: fundamentals

Question	Strong answer
What is Kafka?	A distributed event streaming platform that stores and serves durable streams of records.
Why Kafka?	To decouple services, handle high-throughput streams, support replay, and feed many consumers independently.
What is a topic?	A named stream/category of records, split into partitions.
What is a partition?	An ordered log shard of a topic. It enables scale and parallelism.
Does Kafka guarantee ordering?	Yes within a partition; not globally across all partitions.
What is a broker?	A Kafka server storing and serving partition replicas.

Interview Q&A: production depth

Question	Strong answer
How do you avoid duplicate processing?	Use idempotent consumers, unique keys, transactional/outbox patterns, and careful offset commits.
How do you handle consumer failure?	Kafka rebalances partitions. Restart from committed offsets; duplicates possible, so processing must be idempotent.
How do you handle poison messages?	Retry with backoff, then DLQ with reason, headers, original payload, and replay tooling.
How do you choose partition count?	Based on throughput, ordering requirements, consumer parallelism, future growth, and operational cost.
What causes lag?	Slow consumers, hot partitions, downstream dependency, too few partitions/consumers, rebalances, or broker pressure.
What is exactly-once?	Scoped guarantee using idempotent producers and transactions; external side effects still need idempotency.

Kafka anti-patterns

Kafka is powerful, but misusing it creates expensive distributed confusion.

Anti-pattern	Why it hurts	Better choice
Kafka for request/response RPC	Adds latency/complexity for immediate replies	HTTP/gRPC for sync flows
Huge payloads in events	Broker/network/storage pressure	Store blob elsewhere, send reference
No schema ownership	Breaks consumers silently	Data contracts and compatibility rules
One topic for everything	Messy ownership, security, retention	Domain-oriented topics
Random keys when order matters	Broken per-entity ordering	Stable entity key
No DLQ strategy	Poison messages block progress	Retry topics + DLQ + replay tooling

When not to use Kafka

Good architects know when Kafka is unnecessary.

Situation	Better option
Simple CRUD app with low traffic	Database + normal APIs
Need immediate request/response	HTTP/gRPC
Small background job queue	Redis queue, RabbitMQ, managed task queue
Need ad-hoc querying over current state	Database/search engine
Team cannot operate Kafka safely	Managed Kafka or simpler architecture
No replay, no fanout, no throughput pressure	Do not add Kafka just for fashion

REMEMBER

Kafka is best when you need durable streams, replay, fanout, high throughput, or decoupled event-driven architecture.

Cheat sheet: components

Component	One-liner	Memory image
Producer	Writes events	Dispatcher sending capsules
Topic	Named event stream	Dedicated highway lane
Partition	Ordered shard of a topic	Train track
Broker	Stores partition replicas	Station/tower
Consumer	Reads events	Retrieval squad
Consumer group	Team sharing partitions	Workers splitting lanes
Offset	Progress marker	Checkpoint marker
Lag	Distance behind latest event	Uncollected cargo
ISR	Healthy replica set	Backup engines caught up

Cheat sheet: production configs

Area	Configs/concepts to know	Why
Producer reliability	acks, retries, enable.idempotence, delivery.timeout.ms	Controls loss/duplicates/latency
Producer throughput	batch.size, linger.ms, compression.type	Controls batching and wire cost
Consumer progress	enable.auto.commit, max.poll.interval.ms, session.timeout.ms	Controls duplicates, liveness, rebalances
Topic durability	replication.factor, min.insync.replicas, retention.ms	Controls availability and replay
Security	TLS, SASL/mTLS/OAuth, ACLs	Controls access and data exposure
Operations	consumer lag, ISR, disk, broker request latency	Controls alerting and health

Best memory strategy for Kafka

Remember Kafka as a futuristic transport city, not a pile of definitions.

Kafka concept	Visual story hook
Event	A sealed data capsule saying "something happened"
Producer	Dispatcher launching capsules
Topic	Named highway lane
Partition	Parallel train track with strict local order
Broker	Station storing and serving capsules
Consumer group	Robot squad dividing tracks
Offset	Checkpoint marker on the track
Lag	Cargo pile waiting behind the squad
Replication	Backup trains following the leader train

REMEMBER

Visual memory beats term memorization. Learn the story, then map it back to the formal terms.

30/60/90 minute learning plan

Time	Do this
0-30 min	Learn core mental model: event, topic, partition, broker, consumer, offset. Draw architecture once.
30-60 min	Understand ordering, partition keys, consumer groups, lag, rebalancing, replication.
60-90 min	Study delivery semantics, producer acks, retries, idempotency, DLQ, outbox, monitoring.
After 90 min	Build one local producer/consumer demo and intentionally crash/restart consumers to observe duplicates.

REMEMBER

The best practical exercise: publish `OrderPlaced` events, run two consumer groups, stop one consumer, restart it, and observe offsets/lag.

Glossary

Term	Meaning
Append-only log	Data structure where new records are added to the end.
Broker	Kafka server storing topic partitions.
Consumer group	Set of consumers that coordinate to share partitions.
DLQ	Dead letter queue/topic for messages that cannot be processed normally.
ISR	In-sync replicas, replicas caught up enough to be eligible for safe acknowledgements.
KRaft	Kafka metadata quorum mode replacing ZooKeeper in modern Kafka.
Offset	Position of a record within a partition.
Partition	Ordered shard of a topic.
Rebalance	Redistribution of partitions across consumers in a group.
Topic	Named stream/category of records.

References and further reading

Apache Kafka Documentation
<https://kafka.apache.org/documentation/>

Confluent - Introduction to Apache Kafka

Sources used to cross-check Kafka definitions, architecture, consumer design, replication, security, monitoring, and modern KRaft notes. Accessed May 2026.
<https://docs.confluent.io/kafka/introduction.html>

Confluent - Kafka Replication and Committed Messages

<https://docs.confluent.io/kafka/design/replication.html>

Confluent - Kafka Consumer Design: Consumers, Consumer Groups, and Offsets

<https://docs.confluent.io/kafka/design/consumer-design.html>

Confluent Developer - Intro to Kafka Partitions

<https://developer.confluent.io/courses/apache-kafka/partitions/>

Confluent Developer - Kafka Transactions and Exactly-Once Semantics

<https://developer.confluent.io/courses/architecture/transactions/>

Apache Kafka - Security Overview

<https://kafka.apache.org/41/security/security-overview/>

Apache Kafka - Monitoring

<https://kafka.apache.org/41/operations/monitoring/>

Apache Kafka - Upgrading to Kafka 4.0 / KRaft notes

<https://kafka.apache.org/40/getting-started/upgrade/>

Confluent Blog - Exactly-Once Semantics Are Possible

<https://www.confluent.io/blog/exactly-once-semantics-are-possible-heres-how-apache-kafka-does-it/>